

Operating Systems Lab Experiments - Aims & Source Codes

Experiment No. 2

Aim

Write shell scripts to display OS version, release number, kernel version, current logged in user, shell, home directory, OS type, path, current working directory, top 10 processes, and processes with highest memory usage.

Source Code (Bash/Shell Script)

```
#!/bin/bash
echo "--- System Information ---"
echo "Kernel release number: $(uname -r)"
echo "Kernel version: $(uname -v)"
echo "Current user: $(whoami)"
echo "Logname: $(logname)"
echo "Current shell: $SHELL"
echo "Home directory: $HOME"
echo "Operating system type: $OSTYPE"
echo "Current path setting: $PATH"
echo "Current working directory: $(pwd)"

echo -e "
--- Top 10 processes in descending order (by CPU) ---"
ps -eo pid,comm,%cpu --sort=-%cpu | head -11

echo -e "
--- Processes with highest memory usage ---"
ps -eo pid,comm,%mem --sort=-%mem | head -11
```

Experiment No. 3

Aim

Create a child process in Linux using `fork()` system call. From the child process obtain the process ID of both child and parent by using `getpid()` and `getppid()` system call. Explore `wait()` and `waitpid()` before termination of process.

Source Code (C)

```
#include <stdio.h>
```

```

#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t parent_ID = getpid();
    pid_t child_ID = fork();

    if (child_ID < 0) {
        printf("Fork failed
");
        return 1;
    } else if (child_ID == 0) {
        // Child process execution
        printf("Child Process -> PID: %d, Parent PID: %d
", getpid(), getppid());
        for (int i = 1; i <= 3; i++) {
            printf("Child counting: %d
", i);
            sleep(1);
        }
    } else {
        // Parent process execution
        printf("Parent Process -> PID: %d, Child PID: %d
", parent_ID, child_ID);
        wait(NULL); // Wait for the child process to terminate
        printf("Child process terminated. Parent resuming and exiting.
");
    }
    return 0;
}

```

Experiment No. 4

Aim

Write a program to demonstrate the concept of non-preemptive CPU scheduling algorithms (First-Come, First-Served - FCFS, and Shortest Job First - SJF).

Source Code (C++)

```

#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

struct Process {
    int id, bt, wt, tat;

```

```

};

bool compareSJF(Process a, Process b) {
    return a.bt < b.bt;
}

void calculateTimes(vector<Process>& p) {
    int n = p.size();
    p[0].wt = 0;
    p[0].tat = p[0].bt;
    float avg_wt = 0, avg_tat = p[0].tat;

    for (int i = 1; i < n; i++) {
        p[i].wt = p[i - 1].wt + p[i - 1].bt;
        p[i].tat = p[i].wt + p[i].bt;
        avg_wt += p[i].wt;
        avg_tat += p[i].tat;
    }

    cout << "Process    BT    WT    TAT
";
    for (int i = 0; i < n; i++)
        cout << "P" << p[i].id << " " << p[i].bt << " " << p[i].wt << "
        " << p[i].tat << "
";
    cout << "Average WT: " << avg_wt / n << "
Average TAT: " << avg_tat / n << "

";
}

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    vector<Process> p(n);
    for (int i = 0; i < n; i++) {
        p[i].id = i + 1;
        cout << "Burst time for P" << i + 1 << ": ";
        cin >> p[i].bt;
    }

    cout << "
--- FCFS Scheduling ---
";
    vector<Process> fcfs = p;
    calculateTimes(fcfs);

```

```

        cout << "--- SJF Scheduling ---
";
        vector<Process> sjf = p;
        sort(sjf.begin(), sjf.end(), compareSJF);
        calculateTimes(sjf);

        return 0;
}

```

Experiment No. 5

Aim

Write a program to demonstrate the concept of preemptive scheduling algorithms (Preemptive Priority Scheduling and Round Robin).

Source Code (C)

```

#include <stdio.h>

void roundRobin() {
    int n, tq, time = 0, remain, i;
    int bt[20], rt[20], wt = 0, tat = 0;

    printf("
--- Round Robin Scheduling ---
Enter number of processes: ");
    scanf("%d", &n);
    remain = n;

    for (i = 0; i < n; i++) {
        printf("Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    printf("
Process      WT      TAT
");
    for (i = 0; remain != 0;) {
        if (rt[i] <= tq && rt[i] > 0) {
            time += rt[i];
            rt[i] = 0;
            remain--;
            printf("P%d %d      %d
", i + 1, time - bt[i], time);

```

```

        wt += time - bt[i];
        tat += time;
    } else if (rt[i] > 0) {
        rt[i] -= tq;
        time += tq;
    }
    if (i == n - 1) i = 0;
    else i++;
}
printf("Avg WT: %.2f
Avg TAT: %.2f
", (float)wt / n, (float)tat / n);
}

void priorityScheduling() {
    int n, i, time = 0, completed = 0, highest;
    int at[20], bt[20], rt[20], pr[20], wt = 0, tat = 0;

    printf("
--- Preemptive Priority Scheduling ---
Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter Arrival Time, Burst Time, Priority for P%d: ", i
+ 1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
        rt[i] = bt[i];
    }

    printf("
Process      WT      TAT
");
    while (completed != n) {
        highest = 9999;
        int pos = -1;
        for (i = 0; i < n; i++) {
            if (at[i] <= time && rt[i] > 0 && pr[i] < highest) {
                highest = pr[i];
                pos = i;
            }
        }
        if (pos == -1) {
            time++;
            continue;
        }
        rt[pos]--;
        time++;
        if (rt[pos] == 0) {

```

```

        completed++;
        int finish = time;
        int waitTime = finish - bt[pos] - at[pos];
        if (waitTime < 0) waitTime = 0;
        printf("P%d %d    %d\n", pos + 1, waitTime, finish - at[pos]);
        wt += waitTime;
        tat += finish - at[pos];
    }
}
printf("Avg WT: %.2f\nAvg TAT: %.2f\n", (float)wt / n, (float)tat / n);
}

int main() {
    roundRobin();
    priorityScheduling();
    return 0;
}

```

Experiment No. 6

Aim

Write a C program to implement the solution of the Producer-Consumer problem using Semaphores.

Source Code (C)

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void* producer(void* arg) {
    for (int i = 0; i < 5; i++) {
        int item = rand() % 100;
        sem_wait(&empty);

```

```

        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Produced: %d at position %d
", item, in);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
    pthread_exit(NULL);
}

void* consumer(void* arg) {
    for (int i = 0; i < 5; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumed: %d from position %d
", item, out);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        sleep(2);
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}

```

```
}
```

Experiment No. 7

Aim

Write a program to demonstrate the concept of deadlock avoidance through Banker's Algorithm.

Source Code (C)

```
#include <stdio.h>

int main() {
    int n, m, i, j, k;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[10][10], max[10][10], need[10][10], avail[10];

    printf("
Enter Allocation Matrix:
");
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("
Enter Max Matrix:
");
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
            need[i][j] = max[i][j] - alloc[i][j];
        }

    printf("
Enter Available Resources:
");
    for (i = 0; i < m; i++)
        scanf("%d", &avail[i]);

    int finish[10] = {0}, safeSeq[10], count = 0;

    while (count < n) {
        int found = 0;
        for (i = 0; i < n; i++) {
```



```

        if (finish[i] == 0) {
            int flag = 0;
            for (j = 0; j < m; j++) {
                if (need[i][j] > avail[j]) {
                    flag = 1;
                    break;
                }
            }
            if (flag == 0) {
                for (k = 0; k < m; k++)
                    avail[k] += alloc[i][k];
                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
        if (found == 0) {
            printf("
System is NOT in a safe state (Deadlock possible)
");
            return 0;
        }

        printf("
System is in SAFE state
Safe sequence is: ");
        for (i = 0; i < n; i++)
            printf("P%d ", safeSeq[i]);
        printf("
");

        return 0;
    }
}

```

Experiment No. 8

Aim

Write a program to demonstrate the concept of dynamic partitioning placement algorithms: Best Fit, First Fit, and Worst-Fit.

Source Code (C)

```

#include <stdio.h>

void firstFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[n], tempBlocks[m];

```

```

for (int i = 0; i < n; i++) allocation[i] = -1;
for (int i = 0; i < m; i++) tempBlocks[i] = blockSize[i];

for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (tempBlocks[j] >= processSize[i]) {
            allocation[i] = j;
            tempBlocks[j] -= processSize[i];
            break;
        }
    }
}

printf("
--- First Fit ---
Process No. Process Size      Block No.
");
for (int i = 0; i < n; i++) {
    printf("%d          %d          ", i + 1, processSize[i]);
    if (allocation[i] != -1) printf("%d
", allocation[i] + 1);
    else printf("Not Allocated
");
}

}

void bestFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[n], tempBlocks[m];
    for (int i = 0; i < n; i++) allocation[i] = -1;
    for (int i = 0; i < m; i++) tempBlocks[i] = blockSize[i];

    for (int i = 0; i < n; i++) {
        int bestIdx = -1;
        for (int j = 0; j < m; j++) {
            if (tempBlocks[j] >= processSize[i]) {
                if (bestIdx == -1 || tempBlocks[j] <
tempBlocks[bestIdx])
                    bestIdx = j;
            }
        }
        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            tempBlocks[bestIdx] -= processSize[i];
        }
    }

    printf("
--- Best Fit ---
Process No. Process Size      Block No.
");
    for (int i = 0; i < n; i++) {

```

```

        printf("%d          %d          ", i + 1, processSize[i]);
        if (allocation[i] != -1) printf("%d
", allocation[i] + 1);
        else printf("Not Allocated
");
    }
}

void worstFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[n], tempBlocks[m];
    for (int i = 0; i < n; i++) allocation[i] = -1;
    for (int i = 0; i < m; i++) tempBlocks[i] = blockSize[i];

    for (int i = 0; i < n; i++) {
        int worstIdx = -1;
        for (int j = 0; j < m; j++) {
            if (tempBlocks[j] >= processSize[i]) {
                if (worstIdx == -1 || tempBlocks[j] >
tempBlocks[worstIdx])
                    worstIdx = j;
            }
        }
        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            tempBlocks[worstIdx] -= processSize[i];
        }
    }
    printf("
--- Worst Fit ---
Process No. Process Size      Block No.
");
    for (int i = 0; i < n; i++) {
        printf("%d          %d          ", i + 1, processSize[i]);
        if (allocation[i] != -1) printf("%d
", allocation[i] + 1);
        else printf("Not Allocated
");
    }
}

int main() {
    int blockSize[] = {6, 12, 25, 20};
    int processSize[] = {10, 20, 5, 15};
    int m = sizeof(blockSize) / sizeof(blockSize[0]);
    int n = sizeof(processSize) / sizeof(processSize[0]);

    firstFit(blockSize, m, processSize, n);
    bestFit(blockSize, m, processSize, n);
    worstFit(blockSize, m, processSize, n);
}

```

```
    return 0;
}
```

Experiment No. 9

Aim

Write a program in C to demonstrate the concept of page replacement policies for handling page faults: FIFO and LRU.

Source Code (C)

```
#include <stdio.h>

void fifo(int pages[], int n, int frames) {
    int frame[10], front = 0, faults = 0, found;
    for (int i = 0; i < frames; i++) frame[i] = -1;

    printf("
--- FIFO Page Replacement ---
");
    for (int i = 0; i < n; i++) {
        found = 0;
        for (int j = 0; j < frames; j++) {
            if (frame[j] == pages[i]) { found = 1; break; }
        }
        if (!found) {
            frame[front] = pages[i];
            front = (front + 1) % frames;
            faults++;
        }
    }
    printf("Total Page Faults (FIFO): %d
", faults);
}

void lru(int pages[], int n, int frames) {
    int frame[10], last_used[10], faults = 0, found;
    for (int i = 0; i < frames; i++) { frame[i] = -1; last_used[i] = -1; }

    printf("
--- LRU Page Replacement ---
");
    for (int i = 0; i < n; i++) {
        found = 0;
        for (int j = 0; j < frames; j++) {
            if (frame[j] == pages[i]) {
```

```

        found = 1;
        last_used[j] = i;
        break;
    }
}
if (!found) {
    int lru_idx = 0, min_time = last_used[0];
    for (int j = 1; j < frames; j++) {
        if (last_used[j] < min_time) {
            min_time = last_used[j];
            lru_idx = j;
        }
    }
    frame[lru_idx] = pages[i];
    last_used[lru_idx] = i;
    faults++;
}
}
printf("Total Page Faults (LRU): %d", faults);
}

int main() {
    int pages[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3};
    int n = sizeof(pages)/sizeof(pages[0]), frames = 3;

    printf("Reference String: 7 0 1 2 0 3 0 4 2 3 0 3");
    printf("Number of Frames: %d", frames);

    fifo(pages, n, frames);
    lru(pages, n, frames);

    return 0;
}

```

Experiment No. 10

Aim

Write a C program to simulate File allocation strategies typically sequential, indexed and linked files.

Source Code (C)

```

#include <stdio.h>
#include <stdlib.h>

```

```

void sequential() {
    int file_size = 100, block_size = 10;
    int blocks = (file_size + block_size - 1) / block_size;
    int start;

    printf("
--- Sequential Allocation ---
Enter starting block: ");
    scanf("%d", &start);
    for (int i = 0; i < blocks; i++) {
        printf("Block %d -> %d
", i, start + i);
    }
}

void linked() {
    int file_size = 100, block_size = 10;
    int blocks = (file_size + block_size - 1) / block_size;
    int arr[50];

    printf("
--- Linked Allocation ---
Enter %d block numbers:
", blocks);
    for (int i = 0; i < blocks; i++) scanf("%d", &arr[i]);

    for (int i = 0; i < blocks - 1; i++) {
        printf("%d -> %d
", arr[i], arr[i + 1]);
    }
    printf("%d -> NULL
", arr[blocks - 1]);
}

void indexed() {
    int file_size = 100, block_size = 10;
    int blocks = (file_size + block_size - 1) / block_size;
    int index[50], index_block;

    printf("
--- Indexed Allocation ---
Enter index block number: ");
    scanf("%d", &index_block);
    printf("Enter %d block numbers:
", blocks);
    for (int i = 0; i < blocks; i++) scanf("%d", &index[i]);

    printf("Index Block: %d
", index_block);
}

```

```

        for (int i = 0; i < blocks; i++) {
            printf("Block %d -> %d
", i, index[i]);
        }
    }

int main() {
    int choice;
    while(1) {
        printf("
1. Sequential Allocation
2. Linked Allocation
3. Indexed Allocation
4. Exit
Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: sequential(); break;
            case 2: linked(); break;
            case 3: indexed(); break;
            case 4: return 0;
            default: printf("Invalid choice. Try again.
");
        }
    }
}

```